EnCoDe: Energy Estimation of Source Code At Design-Time

Shailender Goyal
Software Engineering Research Center, IIIT Hyderabad
Hyderabad, India
shailender.goyal@research.iiit.ac.in

Akhila Matathammal
Software Engineering Research Center, IIIT Hyderabad
Hyderabad, India
akhila.matathammal@research.iiit.ac.in

Karthik Vaidhyanathan
Software Engineering Research Center, IIIT Hyderabad
Hyderabad, India
karthik.vaidhyanathan@iiit.ac.in

ABSTRACT

Energy efficiency has emerged as a vital attribute of software quality, with significant implications for both environmental sustainability and operational costs. However, existing profiling tools operate only at runtime and coarse granularity, typically capturing energy at the process or method level. Such tools fail to expose how small code blocks, such as functions, loops, and conditionals, contribute to energy consumption, preventing developers from reasoning about and comparing the energy efficiency of programming constructs during design-time.

To address this gap, we propose EnCoDe, a methodology for fine-grained, design-time energy estimation, with the following key contributions: (1) PowerLens, a novel measurement methodology that achieves reliable sub-millisecond energy readings for small code blocks; (2) Extensive empirical study on code blocks extracted from over 18,000 Python programs, uncovering linear and non-linear relationships between energy consumption and static code features such as structural, complexity, density, and contextual characteristics, resulting in a first-of-its-kind fine-grained dataset; and (3) Predictive modeling, in which machine learning models are trained on these features to accurately estimate and classify block-level energy consumption at design-time. Our results demonstrate stable, reproducible block-level estimations, with regressors achieving $R^2 = 0.75$ and classifiers achieving 80.6% accuracy in identifying energy hotspots, enabling developers to localize and address inefficient code regions early in the development process without execution.

KEYWORDS

Green Software Engineering, Software Sustainability, Design-Time, Energy Estimation, Static Code Analysis

1 INTRODUCTION

Source code is the fundamental component of software systems, defining the behavior of systems ranging from embedded IoT devices to global communication networks [10, 20]. While hardware efficiency has improved significantly, the software running on it remains a massive consumer of energy, often written without regard for its environmental footprint due to software bloat and abstraction layers [23, 41]. The aggregate energy consumption of inefficient code across billions of devices contributes heavily to global carbon emissions [6, 11]. To mitigate this, we must move beyond traditional post-execution profiling and develop predictive models capable of estimating the energy consumption of code blocks **during the development phase**. By quantifying the energy cost of specific constructs before compilation, developers can treat energy as a tangible resource and make informed decisions in real-time, rather than relying solely on retrospective measurements [26, 28].

Despite this need, developing energy-efficient software remains structurally difficult. Current assessment techniques rely predominantly on *runtime measurements* using hardware power meters or processor-level counters like Intel RAPL [1, 12]. This creates a reactive *"measure-after-build"* workflow, where a program must be fully implemented and executed before its energy behavior can be observed. This late-stage feedback is costly; discovering an energy defect after deployment often requires expensive refactoring cycles. Furthermore, existing profiling tools often provide insufficient granularity for effective optimization [22]. They typically report energy usage at the process or application level, failing to attribute computation to specific code constructs such as loops or conditional blocks that developers actively manage [17, 34]. This disconnect leaves developers unable to pinpoint the exact source of energy inefficiencies, reducing optimization efforts to trial and error [28].

To address these limitations, proactive approaches are needed that shift energy assessment from a runtime concern to a design-time quality attribute. Developers currently rely on static analysis tools and linters such as *SonarQube* [19], *PMD* [4], *CheckStyle*[1] , and *SpotBugs*[2] to identify bugs and enforce coding standards before code is even compiled. A similar proactive approach is required for energy efficiency. However, there are insufficient tools that can estimate the energy implications of source code structure during the development phase. To the best of our knowledge, no existing approach provides fine-grained, block-level energy estimation purely from source code at design time without requiring code execution or specialized hardware profiling. This gap forces energy efficiency to remain a retroactive afterthought rather than a proactive design parameter.

In this paper, we propose ***EnCoDe***, a novel methodology for ***"design-time, fine-grained estimation of energy consumption"***. Our methodology bridges the gap between static code structure and dynamic energy behavior. To achieve this, we first address the measurement gap by developing ***PowerLens***, a custom measurement methodology capable of reliably profiling code blocks at sub-millisecond granularity. This infrastructure allows us to construct a ground-truth dataset of code blocks annotated with precise energy consumption values. Leveraging this dataset, we extract rich structural features from ***Abstract Syntax Trees (ASTs)*** and train machine learning models to predict block-level energy consumption statically. We explicitly employ classical, low-overhead machine learning approaches (e.g., Random Forests, Gradient Boosting) rather than energy-intensive deep learning models. This alignment with the principles of Data-Centric Green AI [39] ensures

[1] `Checkstyle`: https://checkstyle.sourceforge.io/
[2] `SpotBugs`: https://spotbugs.github.io/

that our solution does not ironically consume excessive energy in the pursuit of energy efficiency, maintaining a low carbon footprint for the tool itself.

The specific contributions of this paper are as follows:

(1) ***PowerLens Measurement:*** We present a novel measurement methodology that utilizes execution amplification and temporal synchronization to **achieve reliable sub-millisecond energy readings**. This allows us to accurately profile microsecond-scale code blocks that are otherwise invisible to standard hardware counters.
(2) ***Fine-Grained Energy Dataset:*** We conducted an extensive empirical study on executable code blocks extracted from over **18,000 Python programs**. This study uncovers the linear and non-linear relationships between static code features such as structural complexity, operator density, and energy consumption.
(3) ***Predictive Modeling and Validation:*** We demonstrate that classical machine learning models trained on these static features can accurately estimate energy consumption at development time. Our results show that EnCoDe achieves a coefficient of determination R^2 **of 0.75** for regression and an **accuracy of 80.6%** for classifying energy hotspots, enabling developers to identify and address inefficiences early in the software development lifecycle.

The remainder of this paper is structured as follows. **Section 2** reviews the related work in software energy measurement and static analysis. **Section 4** detailed the EnCoDe methodology, including the PowerLens methodology and the feature extraction process. **Section 5** describes the experimental setup and dataset construction. **Section 6** formulates the research questions guiding this study. **Section 7** presents our experimental results and analysis. **Section 8** discusses the implications of our finding.**section 9** addresses the threats to validity. Finally, **Section 10** documents our conclusions and outlines future work.

2 RELATED WORK

Research into software energy efficiency has primarily focused on two distinct areas: runtime measurement infrastructures and predictive modeling for specific domains.

2.1 Software Energy Measurement

Accurate energy measurement is the prerequisite for optimization. The most reliable method involves external hardware power meters, but their high cost and non-scalability limit their use to lab environments [14]. Consequently, the research community has shifted toward software-base power models. Intel's RAPL (Running Average Power Limit) has become the de facto standard for accessing on-chip energy counters [17]. However, as noted in recent studies, RAPL's update frequency (approx. 1ms) is often too coarse to capture the energy consumption of short-lived code constructs [12]. To address this granularity issue, tools like ALEA [22] and finer-grained profilers for embedded systems has been proposed. These approaches often use probabilistic sampling or instruction-level characterization to attribute energy to specific code regions.

2.2 Static Analysis and Energy Prediction

To move energy assessment earlier in the lifecycle, researchers have explored static analysis and machine learning. In the mobile domain, approaches like *MLEE*[5] and *EcoAndroid* [33], have successfully demonstrated that structural metrics can predict energy consumption for Android applications. Similarly, *FlipFlop* [30] utilizes static analysis to optimize GPU kernel configurations for AI workloads.

For general-purpose languages like Python, tools such as *GreenPy* [32] provide application-level energy insights. However, these existing approaches typically operate at the level of entire applications, methods, or specific API calls. They lack the resolution to estimate the energy cost of fundamental control flow blocks (e.g., individual `for` loops or `if` conditions) in general-purpose context. Furthermore, many existing models can rely on deep learning, which can itself be energy-intensive. By contrast, EnCoDe targets this specific granularity gap using lightweight, interpretable models compliant with Green AI principles.

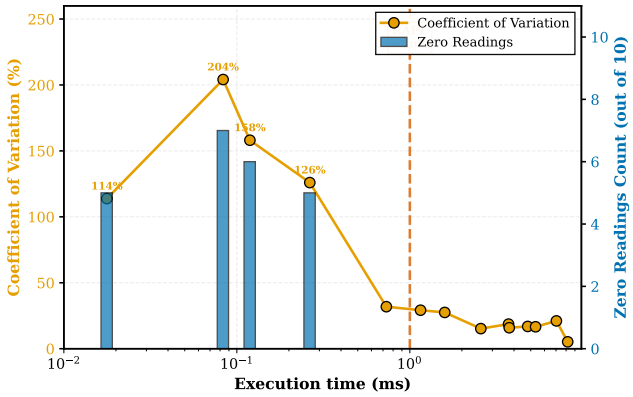

Figure 1: Quality of RAPL's Measurements over Execution Time of the Workload (ms in Log scale)

3 MOTIVATION

Developing energy efficient software would require reliable insight into how individual fragments of source code contribute to energy consumption. Existing measurement techniques use hardware based counters such as Intel's Running Average Power Limit (RAPL)[1]. While effective for coarse grained profiling, but RAPL's millisecond resolution is too large for such code fragments. As Figure 1 demonstrates that workloads under 1ms register more than 110% variation and 4-5 runs get zero readings (out of 10 runs). Even above 1ms, the variation is significant. Although hardware power meters provide higher resolution, they are severely limited in terms of accessibility, reproducibility, and usability in distributed systems. To address this fundamental limitation, we developed **PowerLens**, a methodology for obtaining precise and reproducible energy readings at sub-millisecond granularity. However, accurate measurements alone are insufficient; developers need energy information before code execution. To enable this, we propose **EnCoDe**, which trains machine learning models on PowerLens measurements to predict block-level energy consumption directly from source code

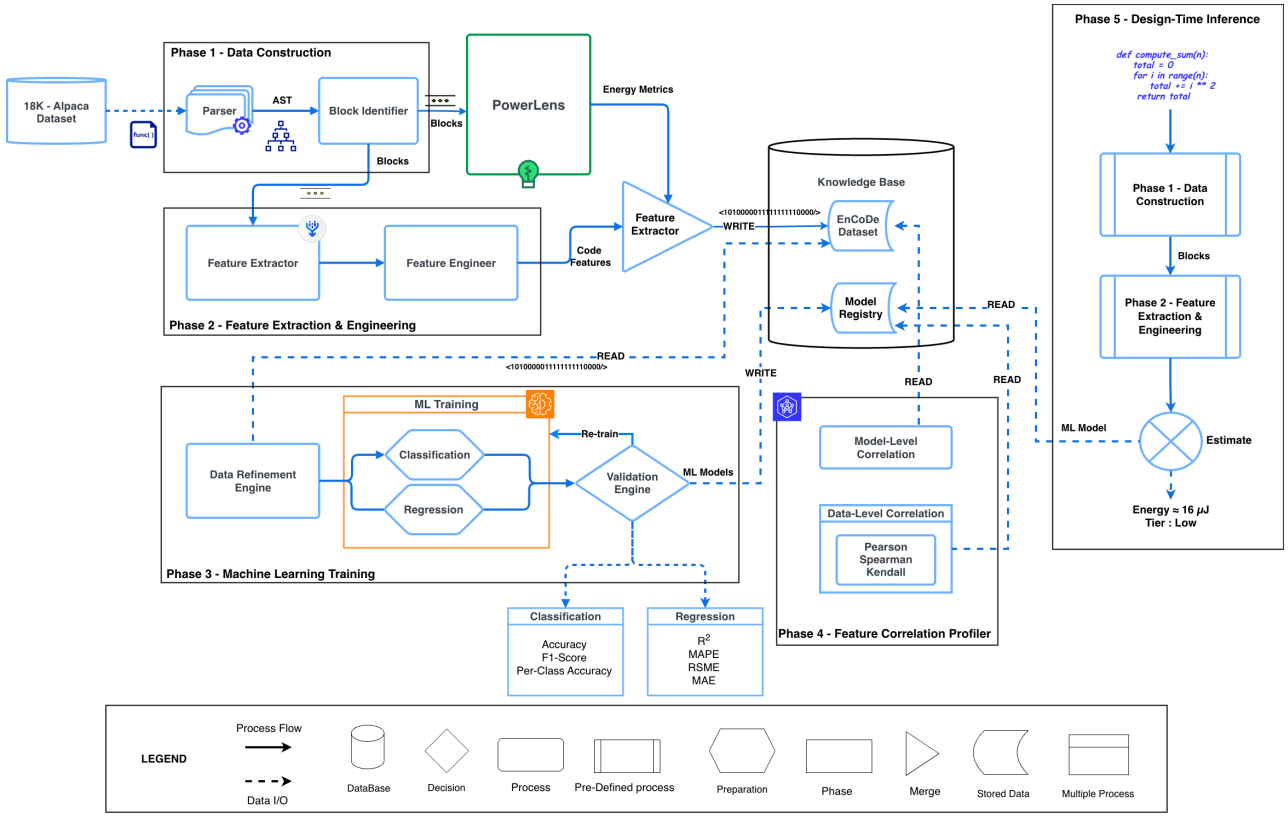

Figure 2: Design Time Energy Estimation Methodology

features, shifting energy assessment from a runtime profiling concern to a proactive design-time estimation.

4 METHODOLOGY

This section details the methodology leveraged by *EnCoDe*. It is organized into a sequence of phases, beginning with the creation of a ground-truth dataset via our novel *PowerLens* measurement methodology, followed by feature extraction, and culminating in the training of predictive models.

To further explain the methodology followed in detail, we will use the following simple running example of a Python function (*Listing 1*), demonstrating how it is transformed from raw code into a final energy estimate.

Example: score_function.py

```python
def calculate_score(items, threshold):
    score = 0
    for item in items:
        if item > threshold:
            score += (item * 2)
    return score
```

Listing 1: Example: score computation function in Python

4.1 Phase 1 - Data Construction

The first phase of our methodology, as shown in *Figure 2*, transforms a raw program text into a structured, analyzable computational block. This is achieved by first parsing the source code $\mathcal{P}$ to its **Abstract Syntax Tree (AST)**, a hierarchical representation of the code's structure[2, 37], as shown in *Figure 3*. We then traverse the AST to identify node types that represent the root of distinct blocks. We define these block-rooting node types as:

$$\mathcal{V}_{block} = \{FunctionDef, For, While, If, Try, With\}$$

Each node of these types, along with the entire subtree rooted at that node, is designated as a distinct block, as shown in *Figure 3*.

For example, after parsing our *Listing 1* into its AST, this phase identifies all nodes whose types match our defined set of block-rooting nodes ($\mathcal{V}_{block}$). This process yields three distinct, nested blocks, each corresponding to a specific AST subtree:

- **B0**: Root node of the AST (module level)
  - **B1**: *FunctionDef*(`calculate_score`)
    - **B2**: *For* loop inside the function
      - **B3**: *If* statement inside the loop

The output of this phase is a collection of AST subtrees, each treated as an atomic block. Crucially, their hierarchical context is preserved (e.g., we know B3 is nested within B2), which is essential for all subsequent measurement and feature extraction.

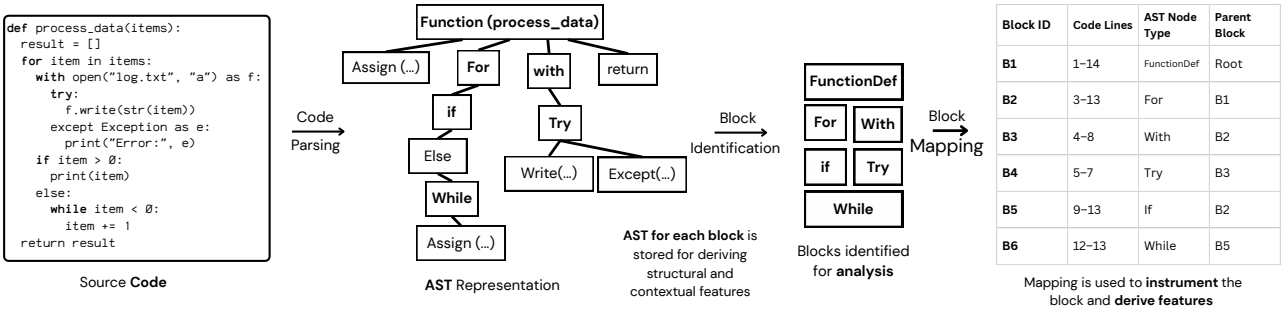

Figure 3: Code Parsing to identify blocks from AST

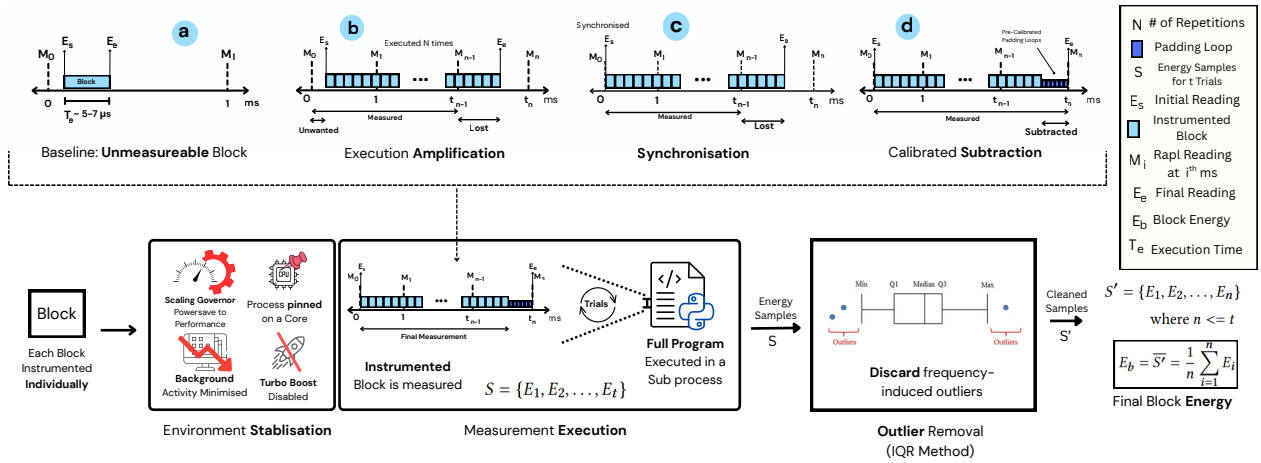

Figure 4: PowerLens : Sub-Millisecond Energy Measurement Methodology

4.2 PowerLens: Fine-Grained Energy Measurement

With the code blocks identified, *PowerLens* measures their energy consumption to create our ground-truth dataset. This is the core technical challenge of our work: a block like the *if* statement (B3) as mentioned in our phase 1, executes in microseconds, far too fast for standard hardware counters like Intel's RAPL to measure directly [1, 12]. Existing tools, therefore, cannot provide this data, as discussed in related work. *PowerLens* addresses this fundamental measurement problem by extending prior work [12], operating through four coordinated mechanisms.

4.2.1 Environmental Stabilization. To ensure measurements are stable, we must isolate the execution from system "noise". Fluctuations in CPU frequency or background processes can easily distort the tiny energy signature. *PowerLens* enforces a stable environment by setting frequency scaling governor to performance, disabling Turbo Boost, pinning the measurement process to a single core, and suspending non-essential background processes. This ensures that the energy we measure is attributable to the code block itself.

For example, before any measurement of B1, B2, or B3 as in *Listing 1* can begin, this stable environment is established. This ensures that any energy we measure is attributable to the code itself, not random system events.

4.2.2 Execution Amplification. A single execution of a block like our *if* statement (B3) is too fast to register on RAPL's millisecond-scale counters. To make this imperceptible signal measurable, *PowerLens* wraps the target block in a tight loop and executes it N times. This amplifies the total energy consumed to a level that rises clearly above the millisecond-scale. The total energy E_{total}, is calculated by taking two snapshots of the RAPL energy counter:(E_s) taken before the execution window, and (E_e) taken immediately after, as shown in *Figure 4*. The measured difference is modeled as:

$$E_{total}(b, N) = E_e - E_s = N * E(b) + \epsilon_N$$

where E_b is the true energy of a single execution and ϵ_N is the measurement noise which becomes negligible when averaged over N repetitions. By linear aggregation, the estimated mean energy per execution, $E(\hat{b})$, becomes:

$$E(\hat{b}) = \frac{E_{total}(b, N)}{N}$$

For example, to measure B3 as discussed in Phase 1, we execute it 1000 times. If the total measured energy E_{total} is 0.04 Joules (J), our initial measurement for a single execution is $\hat{E}(B3) = 0.04J/1000 = 4.0 * 10^{-5}J$.

4.2.3 Synchronization - Aligning Execution and Sampling. Amplification alone is insufficient. If the execution loop starts or end in the middle of a RAPL sampling window, the measurement will be corrupted by energy from unrelated computations, as illustrated by the "unwanted" energy in *Figure 4*. To solve this, *PowerLens* synchronizes the start of the execution loop with the RAPL counter's refresh cycle. The start time t_0 is formally defined as the first moment a counter update is detected after the previous measurement start t_s:

$$t_0 = min\{t > t_s | M_{i+1} > M_i\}$$

where M_i is the cumulative RAPL reading time at time tick *i*. This precise alignment guarantees that the captured energy is attributable only to our target block.

For example, when measuring the amplified loop for B3 as discussed in Phase 1, *PowerLens* waits for the RAPL counter value to change from M_i to M_{i+1} before starting the 1000 executions. This guarantees that the measured E_{total} is not contaminated by prior processes.

4.2.4 Calibrated Subtraction. As amplified execution rarely finishes exactly on a RAPL boundary, a small "padding" interval is needed to complete the measurement window, and this padding consumes energy that must be removed. *PowerLens* subtracts a pre-calibrated energy cost for this padding, $E_{pad}(\delta t)$, which is modeled as a linear function of the padding duration δt. The final, net energy for a single trial is then:

$$E_{net}(b) = \frac{E_{total}(b, N) - E_{pad}(\delta t)}{N}$$

4.2.5 Statistical Aggregation - Achieving Reproducibility. To ensure the final value is robust against any residual system noise (eg., transient interrupts, SMM), the entire process is repeated $m(= 10)$ times. Outliers are filtered from these trials using the standard *Inter-quartile Range (IQR)* rule [38]. The final energy signature for a block, $\hat{E(b)}$, is the mean of the stable, non-outlier observations:

$$\bar{E}(b) = \frac{1}{|\Omega(b)|} \sum_{i \in \Omega(b)} E_{net}^{(i)}(b)$$

where $\Omega(b)$ denotes the non-outlier trial set.

This is applied independently to each block, producing reliable, ground-truth energy values: $\{B1 : 7.7e{-}2J, B2 : 7.5e{-}2J, B3 : 1.4e{-}6J\}$. This data, which could not be obtained with prior software methods forms our ground truth.

4.3 Phase 2 - Feature Extraction and Engineering

After energy measurements, the next step is to create a numerical "fingerprint" for each block based on its source code[31]. The goal is to compute a set of static features that capture the block's structural and syntactic characteristics, which we hypothesize are predictive of its energy consumption. This process creates the analytical foundation for our predictive models, as illustrated in *Figure 2*.

4.3.1 Feature Extractor. This component operates on the AST of each block, computing a feature vector x_b composed of 33 metrics grouped into seven categories. These categories are designed to capture different facets of code:

Basic Metrics (5): Capture the block's overall scale (e.g., AST node count, depth). These serve as a baseline proxy for the total number of components.

Complexity Metrics (4): Quantify logical intricacy using measures like cyclomatic complexity[21]. Prior studies have established a direct correlation between high complexity and increased power consumption, as it often translates to more conditional branches in the instruction stream [16].

Density Metrics (5): Measure the concentration of computational work, such as the ratio of operators to total AST nodes. Dense code regions often correspond to higher instruction throughput and sustained CPU activity.

Diversity Metrics (6): Assess the heterogeneity of code elements (e.g., entropy of operator types). High diversity may engage a wider range of CPU functional units, influencing micro-architectural energy states.

Structural Metrics (3): Capture the shape of the control-flow graph (e.g., branching factor), which impacts instruction fetch and branch prediction energy costs. [3]

Code Pattern Metrics (5): Identify the presence of semantic constructs like loops and conditionals, which are primary determinants of a block's dynamic execution behavior.

Halstead Metrics (5): Model the informational complexity of code (e.g., program volume, program effort)[13]. While designed for cognitive effort, Halstead metrics are considered key indicators of software maintainability, a crucial aspect of overall software sustainability and energy efficiency [25].

For example, the *if* statement (B3) is converted into a feature vector. Illustrative values might include *ASTDepth* (3), *Cyclomatic Complexity* (2), *Operator Density* (0.4), and a binary flag *HasConditional* (1).

4.3.2 Feature Integrator. This final component unifies the outputs of the previous phases to produce the EnCoDe Dataset, a key research artifact of this work, as illustrated in *Figure 2*. The integrator aligns the feature vector x_b with the corresponding energy label $\hat{E}(b)$ for every block.
For example, the feature vector for the if statement (B3) is paired with its measured energy value $\bar{E}$(B3) to create a single data point in our dataset: $(x_B3, 1.4e - 6J)$.

4.4 Phase 3 - Machine Learning Training

With the finalized EnCoDe dataset, the objective of this phase is to train predictive models that can learn the relationships between a block's static features and its measured energy consumption. We frame this as a supervised learning problem, training two complementary models:
Regression Model (M_r)**:** This model learns to predict the continuous energy value (in Joules). This allows to precisely compare the expected energy cost of different implementations.
Classification Model (M_c)**:** This model provides more intuitive classification into energy tiers. The ground-truth energy values are

discretized into three tiers ("Low," "Medium," "High") using equal frequency binning. This provides "lint-like" warning to quickly draw attention to potential energy hotspots.

To ensure robustness and prevent bias from the skewed energy distribution, models are trained using stratified k-fold cross-validation and checked for overfitting.

For our example: The data points for our three blocks: $(x_{B1}, 7.7e{-}2J)$, $(x_{B2}, 7.5e{-}2J)$, and $(x_{B3}, 1.4e{-}6J)$, are fed into the training process. The models learn, for instance, that feature vectors with $HasLoop = 1$ and high $OperatorDensity$ (like x_{B2}) tend to have higher energy values, while simple conditional blocks (like x_{B3}) have very low energy.

4.5 Knowledge Base

The Knowledge Base is the conceptual component in our methodology that stores analytical artifacts, decoupling resource-intensive training from real-time inference. The Knowledge Base logically contains two primary artifacts, as shown in *Figure 2*:

The EnCoDe Dataset (D_{EnCoDe})**:** The complete, finalized mapping of feature vectors to their ground-truth energy values.

The Model Registry: The trained and validated predictive models: a regression model (M_r) for estimating continuous energy values and a classification model (M_c) for predicting discrete energy tiers.

For our example: The data point $(x_B3, 1.4e-6J)$ for our if statement, along with the data for B1 and B2, is stored in the dataset. The trained models, M_r and M_c, which have learned from these and thousands of other examples, are stored in the registry.

4.6 Phase 4 - Feature Correlation Profiler

The purpose of this phase is to provide crucial interpretability of our prediction models. To achieve this, we perform a dual analysis that cross-validates our findings: one perspective is derived from the trained model's behavior, and the other from direct statistical evidence in the data. **Model-Based Feature Importance:** First, we analyze the trained models to rank features based on how much they contribute to prediction accuracy. For tree-based ensembles, a feature's influence $I(f_j)$ can be quantified as its average marginal reduction in prediction error across all decision trees in the model :

$$I(f_j) = \mathbb{E}_{f \in \mathcal{F}}[\Delta Err(f_j)]$$

Data-Level Correlation: Second, we perform a model-agnostic statistical analysis where we compute the correlation coefficients between features and the measured energy values to capture different types of relationships: Pearson (ρ_p) for linear [27], Spearman (ρ_s) for monotonic [36], and Kendall (ρ_k) for rank-based associations[15]. This collection of correlations is represented as:

$$C = \{(f_j, \rho_p(f_j, E)), (f_j, \rho_s(f_j, E)), (f_j, \rho_k(f_j, E))\}$$

By confirming that both analyses highlight the same key features, we increase our confidence that the models are capturing genuine, underlying relationships between code structure and energy, rather than learning from spurious artifacts.

For our example: This dual analysis provides deep insights.

For the *for* loop (B2), the feature importance might rank the binary feature $HasLoop = 1$ and the $CyclomaticComplexity$ metric very highly. Simultaneously, the data-level analysis would likely show a strong positive Spearman correlation (ρ_s) between these same features and energy across the entire dataset. This convergence gives us high confidence that the model has correctly learned a fundamental truth: loops and complex logic are significant drivers of energy consumption. The profiler's output is a unified ranking of these influential features, bridging the gap between prediction and explanation.

4.7 Phase 5 - Design-Time Inference

This phase operationalizes EnCoDe's goal of design-time energy estimation by translating static code blocks into predictions using pre-trained models. The inference engine takes a newly written or modified code block as input and performs the following steps, as in *Figure 2* :

Block Identification and Feature Extraction: The unseen code is parsed into its AST, and its constituent blocks are identified using the same mechanism as in Phase 1. The Feature Extractor from Phase 2 is then reused to compute the corresponding feature vector, x_b^{new}, for each new block.

Model-Based Prediction: The pre-trained regression (M_r) and classification (M_c) models are retrieved from the Model Registry in the Knowledge Base by the inference engine to generate the predictions.

Formally, the inference process can be represented as a transformation function Φ that maps the new block b_{new} and the stored models to an energy estimate and a tier :

$$\Phi\colon (b_{\text{new}}, M_r, M_c) \rightarrow (\hat{E}_b, T_b) \tag{1}$$

where $\hat{E}_b$ is the predicted energy in Joules and T_b is the predicted tier (e.g., "Low," "Medium," "High").Crucially, this entire process is static and does not require any code execution.

For our example: Imagine a developer is refactoring our Listing 1 function and replaces the for loop (B2) with a while loop. As they finish writing the new while loop block, energy is inferred again. The regression model M_r might predict an energy value of $\hat{E}_b = 6.9e - 2J$. Simultaneously, the classification model M_c might predict the tier T_b = "Medium".

5 EXPERIMENTAL SETUP

All experiments were executed on a dedicated, isolated machine under controlled conditions to maximize measurement stability and reproducibility.

Hardware and system environment. Experiments were performed on an Intel CPU (model: `i7-6700K`; with 16 GB DDR4 RAM, running Pop!_OS 22.04 (64 bit). To reduce measurement noise, we stabilzed the environment as in Section 4.2.1. We loaded the `msr` kernel module to access Model-Specific Registers for RAPL readings.

Software environment. All experiments used Python 3.12. Block extraction relied on the standard `ast` module and custom instrumentation utilities. All code, experiment scripts, and seed values for pseudo-random operations are archived and available in the artifact repository. We set a fixed random seed for all learning experiments to ensure reproducible train/test splits and model initializations.

Dataset. The dataset was constructed from a larger corpus of Python programs[3]. After parsing with `ast`, we extracted executable code blocks defined as function bodies, loop bodies (`for`, `while`), conditional bodies (`if`), exception handlers (`try/except`), and context manager blocks (`with`). We filtered out non-executable blocks as we need ground truth measurements. From **18,612** code files, we extract **14,000+** executable blocks, retaining **8,000+** that exceed 1 µs (measurable with N=1000). Each block was annotated with a set of **33 static features** comprising of AST derived features and standard code properties.

Measurement protocol. Energy readings were taken from processor energy counters exposed via Intel RAPL through MSR access. Specifically, we read the `PACKAGE` domains using the MSR interface. All MSR reads were executed with root privileges and synchronized with our measurement control loop to avoid partial-window reads. Block execution times are typically in the microsecond range, while RAPL updates at millisecond granularity; to bridge this mismatch we used deterministic amplification and synchronization as follows:

Evaluation protocol and statistical analysis. We evaluated both regression and classification formulations. Regression performance metrics include R^2 (variance explained), RMSE (root mean squared error), MAE (mean absolute error), and MAPE (mean absolute percentage error) [7][8]. For classification (energy-tier prediction), we report accuracy (correct predictions), precision (positive prediction accuracy), recall (true positive coverage), F1-score (harmonic mean of precision and recall), and confusion matrices [35]. All reported results include mean and standard deviation across folds.

6 RESEARCH QUESTIONS

To guide our investigation, we formulate the following three research questions. These questions are answered through the analyses presented in Section 7.

RQ1: How can energy consumption of small code blocks be measured accurately and reproducibly? We evaluate the proposed methodology's measurement granularity, stability and effectiveness for small code blocks.

RQ2: What statistical relationships exist between static code features and block-level energy consumption? We investigate whether code structure and metrics are statistically related to energy consumption. This leads to two sub-questions:

- **RQ2a**: Which static code features exhibit associations with block's energy consumption?
- **RQ2b**: What is the nature of these associations between energy consumption and code features?

RQ3: How reliably can code features predict block-level energy consumption? Beyond correlation, we ask whether predictive models trained on static features can predict energy consumption and generalize to unseen code. This leads to two sub-questions:

- **RQ3a:** How accurately can regression models predict absolute block-level energy values?
- **RQ3b:** How effectively can classification models assign blocks to energy tiers (low, medium, high)?

[3]https://huggingface.co/datasets/iamtarun/python_code_instructions_18k_alpaca

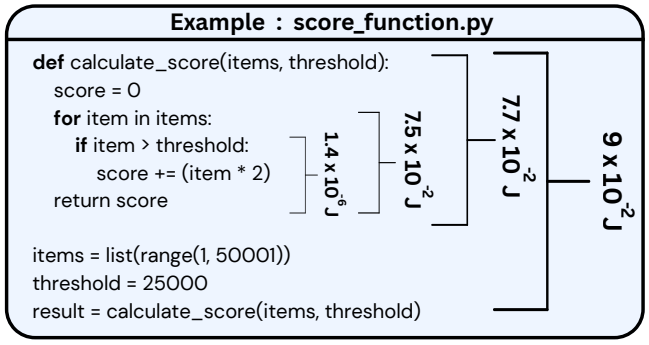

Figure 5: Block Level Energy Measurement of the score computation function

7 RESULTS

We present the results according to the research questions defined in Section 6.

7.1 RQ1: How can energy consumption of small code blocks be measured accurately and reproducibly?

Our first research question examines whether energy consumption of microsecond-scale code blocks can be measured accurately and reproducibly. Each block was measured ten times, and across the dataset **more than 90%** of the blocks exhibited **less than 10% variation** between repeated trials after IQR outlier removal, indicating strong measurement stability and effective suppression of environmental noise. The results demonstrate that the proposed measurement infrastructure provides stable and reliable energy values at block granularity. This is a result of system stabilization and calibrated padding loops.

The measurement protocol also achieves both high sensitivity and broad coverage. The observed energy values spans six orders of magnitude, ranging from 2.37×10^{-5} joules for trivial blocks to 7.48×10^{2} joules for complex functions. This wide dynamic range confirms that the methodology is capable of capturing energy signatures of extremely short-lived constructs while remaining reliable. Without the amplification and synchronization mechanisms, many of these fine-grained blocks would register highly unstable or zero readings using conventional software-profilers.

Figure 5 illustrates block-level energy measurements for the `score_function.py` example, where annotated values indicate per-block energy consumption in joules as measured by PowerLens in Nested Blocks. Figure 6 compares the sum of block-level energy measurements obtained with PowerLens against coarse-grained measurements from PyRAPL[4], grouped by construct type. To **validate** the correctness of the fine-grained measurements, we compared the aggregate energy obtained by summing block-level measurements from PowerLens with the total program-level energy reported by PyRAPL for the same codes, across 40 instances of each block type (see Figure 6). For all block categories, the aggregated PowerLens measurements closely match the corresponding

[4]https://github.com/powerapi-ng/pyRAPL

Table 1: Top 20 features by Correlation and Feature Importance

Rank	Pearson ($\|r\|$)		Spearman ($\|\rho\|$)		Kendall ($\|\tau\|$)		ExtraTrees		Random Forest		Gradient Boosting	
	Feature	Value	Feature	Value	Feature	Value	Feature	Value	Feature	Value	Feature	Value
1	**operator density**	**0.286**	*functions count*	*0.621*	*functions count*	*0.507*	**operator density**	**0.086**	**operator density**	**0.099**	**operator density**	**0.102**
2	**operator entropy**	**0.205**	node type entropy	0.294	node type entropy	0.193	**loops count**	**0.063**	*unique node types*	*0.075*	**program difficulty**	**0.056**
3	conditionals count	0.181	*conditionals count*	*0.229*	*conditionals count*	*0.178*	*functions count*	*0.061*	**program difficulty**	**0.073**	program effort	0.054
4	unique operators	0.178	cognitive complexity	0.225	cognitive complexity	0.165	**operator entropy**	**0.054**	*call density*	*0.072*	*variable entropy*	*0.037*
5	*literal density*	*0.174*	nesting complexity	0.215	unique functions	0.165	*variable entropy*	*0.054*	*variable entropy*	*0.061*	**loops count**	**0.037**
6	functions count	0.166	control flow complexity	0.210	nesting complexity	0.160	*call density*	*0.053*	variable density	0.061	*unique node types*	*0.031*
7	**loops count**	**0.156**	*literal density*	*0.207*	control flow complexity	0.156	**program difficulty**	**0.052**	leaves to nodes ratio	0.053	*call density*	*0.022*
8	*variable entropy*	*0.145*	unique functions	0.205	cyclomatic complexity	0.150	depth variance	0.046	depth variance	0.052	*literal density*	*0.016*
9	variable density	0.144	cyclomatic complexity	0.203	*literal density*	*0.145*	unique variables	0.046	*literal density*	*0.051*	*conditionals count*	*0.014*
10	**program difficulty**	**0.129**	vocabulary size	0.199	vocabulary size	0.141	*unique node types*	*0.043*	program effort	0.042	**operator entropy**	**0.011**
11	unique variables	0.122	**operator density**	**0.195**	**operator density**	**0.138**	literal density	0.042	**operator entropy**	**0.037**	leaves to nodes ratio	0.010
12	leaves to nodes ratio	0.117	*unique node types*	*0.191*	*call density*	*0.133*	*conditionals count*	*0.036*	unique variables	0.032	*functions count*	*0.010*
13	depth variance	0.116	*call density*	*0.183*	*unique node types*	*0.127*	variable density	0.035	unique functions	0.030	program length	0.009
14	attribute density	0.080	program volume	0.148	unique variables	0.110	unique operators	0.032	**loops count**	**0.030**	depth variance	0.009
15	max branching factor	0.073	*variable entropy*	*0.139*	*variable entropy*	*0.105*	unique functions	0.027	total nodes	0.028	unique operators	0.008
16	max depth	0.072	unique variables	0.138	unique operators	0.103	avg depth	0.026	avg depth	0.026	variable density	0.007
17	avg depth	0.071	unique operators	0.138	program volume	0.101	avg branching factor	0.025	node type entropy	0.025	unique functions	0.005
18	cognitive complexity	0.055	program length	0.126	**operator entropy**	**0.094**	leaves to nodes ratio	0.023	unique operators	0.024	program volume	0.004
19	node type entropy	0.048	**operator entropy**	**0.125**	program length	0.087	max depth	0.022	program volume	0.023	unique variables	0.004
20	*unique node types*	*0.047*	leaves to nodes ratio	0.118	leaves to nodes ratio	0.080	cognitive complexity	0.021	program length	0.022	node type entropy	0.003

Bold = linear relation, *italic* = non-linear relation

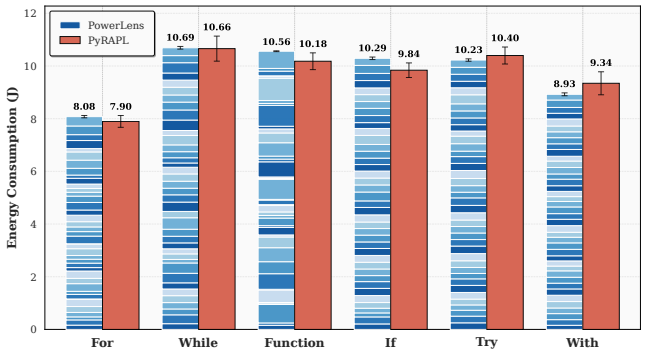

Figure 6: Sum of Blocks measured by Powerlens compared with Overall measurement by Pyrapl

coarse-grained energy reported by PyRAPL. In addition, PowerLens exhibits **substantially lower variance** across repeated measurements, as it stabilizes the execution environment and applies calibration loops to isolate block execution energy.

Answer to RQ1

Together, these results confirm that PowerLens enables accurate, stable, and reproducible energy measurement at the level of individual small code blocks, overcoming the temporal limitations of existing software-based profilers.

7.2 RQ2: What statistical relationships exist between static code features and block-level energy consumption?

We next examine whether code metrics of source code provide meaningful explanatory signals for energy consumption and the nature of these signals.

We present the top 20 features from the correlation analysis, ranked by absolute correlation values (Pearson, Spearman, and Kendall) and feature importance scores (Extra Trees, Random Forest, and Gradient Boosting), in Table 1.

The feature importance results in Table 1 indicate that energy consumption does not strongly depend on any single feature. While several metrics exhibit statistically meaningful correlations with energy, most Pearson correlation coefficients are moderate in magnitude. In contrast, Spearman and Kendall coefficients are substantially higher for many features, suggesting that the relationship between code features and energy consumption is largely **non-linear**.

Features such as `operator_density`, `operator_entropy`, and `loops_count` demonstrate linear associations and consistent predictive power across models. Notably, `operator_density` ranks first in all three models. Other features, including `call_density`, `conditionals_count`, `literal_density`, and `unique_node_types`, exhibit non-linear relationships with energy while maintaining stable predictive importance across models. The convergence of correlation analysis and predictive performance suggests that the models capture genuine underlying relationships.

Although some features achieve high rank-based correlations primarily because they act as categorical indicators, `functions_count` exhibits high Spearman and Kendall coefficients but contributes little to prediction. Such features carry block specific knowledge and hence, effectively distinguish block types but can't predict energy consumption.

Answer to RQ2

Taken together, these findings demonstrate that energy consumption is encoded in the compositional structure of code. Most feature-energy relationships are non-linear, as evidenced by the gap between Pearson and Spearman coefficients. Control flow shape, operator composition, functional decomposition, and nesting patterns jointly determine energy behavior. As a result, features that attempt to explain energy using isolated characteristics are inherently limited. Effective energy modeling requires holistic representations of code capturing feature interactions.

7.3 RQ3: How reliably can code features predict block-level energy consumption?

Table 2: Regression Models with log Transform on Target

Model	Test R^2	CV R^2 (± std)	RMSE	MAE	MAPE	Energy (mJ)
XGBoost	0.755	0.811 ± 0.040	0.281	0.057	172.45	15.26
SVR	0.752	0.803 ± 0.039	0.283	0.091	182.27	15.47
Gradient Boosting	0.747	0.810 ± 0.040	0.286	0.058	171.83	13.56
CatBoost	0.722	0.799 ± 0.043	0.300	0.058	166.35	22.97
Random Forest	0.719	0.793 ± 0.047	0.302	0.060	162.83	258.01
Extra Trees	0.682	0.737 ± 0.047	0.321	0.074	170.85	275.83
Decision Tree	0.648	0.722 ± 0.070	0.338	0.067	170.24	14.69
KNN	0.645	0.684 ± 0.081	0.340	0.066	105.78	19.59
AdaBoost	0.633	0.757 ± 0.051	0.345	0.092	171.81	20.04

Shading indicates top two models by each metric.

Table 3: Energy Tier Classification Models

Model	Accuracy	CV Accuracy	Precision	Recall	F1	Energy(J)
XGBoost	0.806	0.793 ± 0.007	0.804	0.806	0.805	0.022
Random Forest	0.792	0.780 ± 0.007	0.789	0.792	0.789	0.373
Gradient Boosting	0.788	0.794 ± 0.008	0.788	0.788	0.788	0.015
K-NN	0.783	0.771 ± 0.005	0.780	0.783	0.780	0.027
SVM	0.781	0.771 ± 0.009	0.780	0.781	0.778	0.022
Extra Trees	0.769	0.765 ± 0.003	0.768	0.769	0.765	0.315
Decision Tree	0.749	0.736 ± 0.009	0.744	0.749	0.745	0.014
Logistic Regression	0.735	0.726 ± 0.003	0.729	0.735	0.731	0.017
SGD Classifier	0.729	0.713 ± 0.005	0.722	0.729	0.721	0.022
QDA	0.720	0.709 ± 0.010	0.745	0.720	0.711	0.052
LDA	0.717	0.712 ± 0.005	0.710	0.717	0.708	0.017
Ridge Classifier	0.712	0.706 ± 0.006	0.708	0.712	0.700	0.016
Gaussian NB	0.678	0.653 ± 0.002	0.680	0.678	0.669	0.022

Shading indicates top two models by each metric.

We assess prediction accuracy of static features on unseen code blocks and identify effective modeling choices.

As shown in Table 2, ensemble and tree-based regression models achieve strong predictive performance, with the XGBoost regressor attaining the highest test performance (R^2 of 0.755). SVR and Gradient Boosting follow closely, exhibiting similarly high accuracy and stable cross-validation results. These models substantially outperform linear baselines, which struggle to capture the complex feature interactions present in the data. The energy distribution in our dataset is highly skewed, spanning several orders of magnitude. To mitigate this skewness and stabilize learning, we apply square root and logarithm transformations to the target values. These transformations significantly improves model convergence and generalization.

In addition to absolute error metrics, we evaluate relative prediction quality using MAPE. While absolute errors remain small, they are less informative given that the dataset spans several orders of magnitude. MAPE is therefore more appropriate; however, it overstates relative error for extremely low-energy blocks, where even minor absolute deviations result in large relative differences. Among the evaluated models, Random Forest and KNN achieve the lowest MAPE values, whereas Gradient Boosting and XGBoost exhibit higher MAPE values exceeding 1.7. This behavior reflects the intrinsic difficulty of accurately modeling micro-scale energy values and highlights the importance of reporting both absolute and relative error when evaluating fine-grained energy predictors.

The superior performance of ensemble and tree-based models is directly explained by the findings of RQ2. Since energy behavior arises from non-linear interactions among multiple code features, models capable of learning hierarchical decision boundaries and feature compositions are fundamentally better suited for this task. Tree ensembles naturally capture such interactions through recursive partitioning, whereas linear models cannot express these relationships without extensive manual feature engineering.

While ensemble models deliver the highest accuracy, they also need higher modeling energy for inference. This requires a practical energy–accuracy tradeoff: the most accurate predictors might consume more energy themselves. However, this overhead remains negligible compared to the energy savings enabled by design-time optimization. Some models might offer minimal gains in accuracy with huge difference in energy consumption (for e.g. Gradient Boosting over XGBoost) and since the estimation is only supposed to be a guiding direction to identify hotspots, modeling energy to accuracy tradeoff is considerable.

Beyond regression, we evaluate the ability of models to categorize code blocks into energy tiers (low, medium, high) in Table 3. The XGBoost classifier achieves the best performance, with 80.6% accuracy and very stable cross-validation results. These classification results demonstrate that static features support reliable qualitative assessment of energy behavior, which is particularly valuable for developer-facing tools where actionable guidance matters.

Answer to RQ3

Overall, the results demonstrate that block-level energy can be predicted accurately and robustly from static code features. Ensemble models are effective, as they capture the non-linear nature inherent in energy behavior. Lightweight Ensembles would be ideal in balancing modeling energy and performance. Small generalization gaps across folds confirm the stability of the models, with further improvements through targeted feature engineering and hyperparameter tuning.

8 DISCUSSION

8.1 Lessons Learned

Energy can now be treated as a metric of code Quality. Several long-standing challenges identified in prior survey and vision work on energy-aware software engineering can be effectively addressed through fine-grained, design-time energy guidance [18]. The results for **RQ1** and **RQ3** demonstrate that energy consumption can be measured and estimated meaningfully at the level of small code blocks; this granularity aligns with how developers reason about and refactor code. In doing so, our methodology directly responds to earlier calls for improving energy observability beyond system, application and method-level measurements [12, 22]. PowerLens reduces reliance on external hardware power meters to obtain high-resolution measurements. EnCoDe provides a methodology for modeling energy consumption from code structure to inform the software development process.

No single metric is a proxy for Energy. Contrary to approaches that treat some software-quality metrics (for example, size or cyclomatic complexity) as proxies for energy[40], our findings

for **RQ2** indicate that no single metric explains energy consumption to a large extent. Rather, energy behavior emerges from interactions among multiple code properties. The observed correlations and feature importance between block-level energy consumption and code metrics reinforces the view that energy is encoded in the compositional characteristics of code rather than isolated metrics .

Finally, the results for **RQ2** and **RQ3** suggest that energy efficient ensemble models should be used to provide actionable energy guidance. These simple machine learning approaches provide approximate, design-time estimates which can be sufficient to distinguish relatively energy-efficient constructs from energy-intensive ones, enabling useful feedback for developers. This aligns with survey studies that emphasize the practical value of timely but approximate estimation over delayed but precise measurements [18, 44].

8.2 Implications for Practice

From a practitioner's perspective, the key implication of this work is that energy efficiency can be considered meaningfully during code construction rather than relegated to late-stage profiling. By providing energy information at the level of code blocks, EnCoDe aligns with existing developer mental models used during design, code review, and refactoring. Prior studies of developer workflows and tool adoption show that such alignment is critical for uptake of static analysis and quality tools, and design-time energy estimation can follow a similar adoption trajectory [24].

Energy-tier classification evaluated in **RQ3** further lowers the barrier to adoption. Instead of asking developers to interpret raw Joule values or to use specialized profiling hardware, the system surfaces lint-like, actionable information that points to constructs likely to be energy-intensive. This mirrors successful adoption patterns for other static analyzers (for performance and security) and suggests energy concerns can be integrated into routine development practices. Embedding energy awareness in the development lifecycle also aligns with emerging SusDevOps perspectives, where sustainability considerations are continuously addressed alongside performance, reliability, and maintainability [9, 29]. Our empirical results provide initial evidence that such integration is both feasible and useful in practice.

8.3 Implications for Research

For researchers, this work demonstrates how open challenges in energy-aware software engineering can be addressed via an integrated empirical and static approach [18, 43, 44]. By coupling fine-grained runtime measurements with static code features, our methodology operationalizes calls to bridge runtime energy data and design-time artifacts across the software lifecycle. This combined methodology enables several promising research directions: large-scale mining studies across languages and ecosystems; comparative analyzes of energy behavior across programming paradigms (for instance, imperative vs functional styles); and systematic investigations into energy-aware refactoring and transformation strategies.

Beyond methodological advances, our findings lend empirical support to treating energy consumption as a first-class software quality attribute. Prior work has argued conceptually for elevating energy alongside traditional qualities such as performance and maintainability; our fine-grained evidence helps close the gap by showing how energy can be measured, modeled, and reasoned about at the level of individual constructs. Finally, the availability of empirically grounded, block-level datasets opens opportunities to study energy behavior in emerging domains, such as AI-intensive systems and other compute-heavy applications, where understanding fine-grained energy signatures is increasingly critical.

9 THREATS TO VALIDITY

Our study demonstrates the feasibility of fine-grained, design-time energy prediction, but its scope is bounded by some validity concerns which we structure and discuss following the guidelines of Wohlin et al. [42].

Internal validity. Data leakage was minimized by strict train–test separation and stratified 5-fold cross validation was employed. Amplification and Calibration steps assume linear scaling of energy consumption. Overfitting was monitored via cross-validation, with small observed gaps suggesting good generalization. For stability, execution was pinned to a single CPU core, but this limits our ability to capture multi-threading effects.

Construct validity. Our methodology does not model how energy scales with different user inputs, data sizes, or runtime states. Amplification factor (N=1000) may not be ideal for ultra small blocks. Furthermore, RAPL readings are limited to CPU and DRAM power domains, excluding I/O, network activity, or storage. Further, RAPL readings have inherent measurement errors. Our methodology does not model inter-thread interactions or distributed execution patterns.

External validity. Though the dataset is representative, it does not cover the full spectrum of libraries, code styles, or domain specific applications. In addition, all experiments were conducted on a single CPU under Linux. Energy characteristics may differ on ARM processors, GPUs, embedded systems etc. Broader validation across languages and hardware is necessary to establish generality.

10 CONCLUSION AND FUTURE WORK

This paper introduced EnCoDe, a methodology for design-time, fine-grained energy estimation of code blocks, together with PowerLens. PowerLens enables consistent measurement of sub millisecond-scale code blocks. By shifting energy awareness from late-stage profiling to early design, this work supports treating energy as a first-class software quality attribute. A natural next step is to extend beyond Python to other programming languages, runtime environments and modelling techniques, enabling validation across programming paradigms and software stacks. Beyond traditional applications, the same block and component-level analysis can be applied to LLMs and machine learning systems, which consume a lot of energy. Further along, moving from detection to actionable guidance and automated repair will make sustainability effective and accessible.

11 REPRODUCIBILITY STATEMENT

To ensure transparency and reproducibility, we are committed to making our research accessible. All code used to produce the results in this paper is available in the following repository [5].

REFERENCES

[1] [n. d.]. Running Average Power Limit Energy Reporting CVE-2020-8694,... — intel.com. https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/advisory-guidance/running-average-power-limit-energy-reporting.html. [Accessed 19-01-2026].

[2] Alfred V. Aho, Ravi Sethi, and Jeffrey D. Ullman. 1986. Compilers: Principles, Techniques, and Tools. In *Addison-Wesley series in computer science / World student series edition*. https://api.semanticscholar.org/CorpusID:278028060

[3] Miltiadis Allamanis, Earl T. Barr, Premkumar Devanbu, and Charles Sutton. 2018. A Survey of Machine Learning for Big Code and Naturalness. *ACM Comput. Surv.* 51, 4, Article 81 (July 2018), 37 pages. https://doi.org/10.1145/3212695

[4] Eman Abdullah AlOmar, Salma Abdullah AlOmar, and Mohamed Wiem Mkaouer. 2023. On the Use of Static Analysis to Engage Students with Software Quality Improvement: An Experience with PMD. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*. IEEE/ACM, 179–191. https://doi.org/10.1109/ICSE-SEET58685.2023.00023

[5] Hamza Mustafa Alvi, Hammad Majeed, Hasan Mujtaba, and Mirza Omer Beg. 2021. MLEE: Method Level Energy Estimation — A machine learning approach. *Sustainable Computing: Informatics and Systems* 32 (2021), 100594. https://doi.org/10.1016/j.suscom.2021.100594

[6] Lotfi Belkhir and Ahmed Elmeligi. 2018. Assessing ICT global emissions footprint: Trends to 2040 & recommendations. *Journal of Cleaner Production* 177 (2018), 448–463.

[7] Alexei Botchkarev. 2018. Performance Metrics (Error Measures) in Machine Learning Regression, Forecasting and Prognostics: Properties and Typology. *ArXiv* abs/1809.03006 (2018). https://api.semanticscholar.org/CorpusID:52182534

[8] Davide Chicco, Matthijs J. Warrens, and Giuseppe Jurman. 2021. The coefficient of determination R-squared is more informative than SMAPE, MAE, MAPE, MSE and RMSE in regression analysis evaluation. *PeerJ Computer Science* 7 (2021). https://api.semanticscholar.org/CorpusID:236196832

[9] Istvan David. 2025. SusDevOps: Promoting Sustainability to a First Principle in Software Delivery. In *2025 IEEE/ACM 47th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*. 106–110. https://doi.org/10.1109/ICSE-NIER66352.2025.00027

[10] Roberto Di Cosmo and Stefano Zacchiroli. 2017. Software Heritage: Why and How to Preserve Software Source Code. *iPRES 2017 - 14th International Conference on Digital Preservation* (2017). https://hal.archives-ouvertes.fr/hal-01590958

[11] Charlotte Freitag, Mike Berners-Lee, Jeffrey Blue, Eric Tsivian, Chris Kelly, and Nick Berners-Lee. 2021. The real climate and transformative impact of ICT: A critique of estimates, trends, and regulations. *Patterns* 2, 9 (2021), 100340.

[12] Marcus Hähnel, Björn Döbel, Marcus Völp, and Hermann Härtig. 2012. Measuring energy consumption for short code paths using RAPL. *SIGMETRICS Perform. Eval. Rev.* 40, 3 (Jan. 2012), 13–17. https://doi.org/10.1145/2425248.2425252

[13] Maurice H. Halstead. 1977. *Elements of Software Science (Operating and programming systems series)*. Elsevier Science Inc., USA.

[14] A. Hindle. 2014. Green Mining: A Methodology of Relating Software Change and Configuration to Power Consumption. *Journal of Software: Evolution and Process* (2014).

[15] M. G. Kendall. 1938. A New Measure of Rank Correlation. *Biometrika* 30, 1/2 (1938), 81–93. http://www.jstor.org/stable/2332226

[16] C. K. Keong, K. T. Wei, A. A. A. Ghani, and K. Y. Sharif. 2015. Toward Using Software Metrics as Indicator to Measure Power Consumption of Mobile Application: A Case Study. In *Proceedings of the 2015 9th Malaysian Software Engineering Conference (MySEC)*. 172–177.

[17] K. N. Khan, M. Hirki, T. Niemi, J. K. Nurminen, and Z. Ou. 2018. RAPL in Action: Experiences in Using RAPL for Power Measurements. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems* 3, 2 (2018), 1–26. https://doi.org/10.1145/3177754

[18] Sung Une Lee, Niroshinie Fernando, Kevin Lee, and Jean-Guy Schneider. 2024. A survey of energy concerns for software engineering. *Journal of Systems and Software* 210 (2024), 111944. https://doi.org/10.1016/j.jss.2023.111944

[19] Valentina Lenarduzzi, Nyyti Saarimäki, and Davide Taibi. 2020. Some SonarQube issues have a significant but small effect on faults and changes. A large-scale empirical study. *Journal of Systems and Software* 170 (2020), 110750. https://doi.org/10.1016/j.jss.2020.110750

[20] Grace Lewis, Edwin Morris, Dennis Smith, and Lutz Wrage. 2006. Service-Oriented Architecture as an Integration Strategy. *Software Engineering Institute, Carnegie Mellon University* CMU/SEI-2006-TN-007 (2006).

[21] T.J. McCabe. 1976. A Complexity Measure. *IEEE Transactions on Software Engineering* SE-2, 4 (1976), 308–320. https://doi.org/10.1109/TSE.1976.233837

[22] Lev Mukhanov, Dimitrios S. Nikolopoulos, and Bronis R. De Supinski. 2015. ALEA: Fine-Grain Energy Profiling with Basic Block Sampling. In *2015 International Conference on Parallel Architecture and Compilation (PACT)*. 87–98. https://doi.org/10.1109/PACT.2015.16

[23] San Murugesan. 2011. Green Software. *IT Professional* 13, 6 (2011), 64–64. https://doi.org/10.1109/MITP.2011.105

[24] Hira Noman, Naeem Ahmed Mahoto, Sania Bhatti, Hamad Ali Abosaq, Mana Saleh Al Reshan, and Asadullah Shaikh. 2022. An Exploratory Study of Software Sustainability at Early Stages of Software Development. *Sustainability* 14, 14 (2022). https://doi.org/10.3390/su14148596

[25] D. Olvera et al. 2022. A Software Sustainability Assessment Tool Based on ISO 25010. *International Journal of Advanced Science and Engineering Information Technology* 12, 4 (2022), 1729–1736.

[26] Cheng Pang, Abram Hindle, Bram Adams, and Hassan J. Ahmed. 2016. What is the True Cost of Software Energy Consumption?. In *Proceedings of the 13th International Conference on Mining Software Repositories (MSR)*. 258–269.

[27] Karl Pearson. 1896. Mathematical Contributions to the Theory of Evolution. III. Regression, Heredity, and Panmixia. *Philosophical Transactions of the Royal Society of London Series A* 187 (Jan. 1896), 253–318. https://doi.org/10.1098/rsta.1896.0007

[28] Gustavo Pinto and Fernando Castor. 2017. Energy-aware software development: A survey. *IEEE Software* 34, 3 (2017), 28–39.

[29] Saurabhsingh Rajput, Tim Widmayer, Ziyuan Shang, Maria Kechagia, Federica Sarro, and Tushar Sharma. 2024. Enhancing Energy-Awareness in Deep Learning through Fine-Grained Energy Measurement. *ACM Trans. Softw. Eng. Methodol.* 33, 8, Article 211 (Dec. 2024), 34 pages. https://doi.org/10.1145/3680470

[30] Saurabhsingh Rajput, Tim Widmayer, Ziyuan Shang, Maria Kechagia, Federica Sarro, and Tushar Sharma. 2024. FlipFlop: A Static Analysis-based Energy Optimization Framework for GPU Kernels. Preprint available at https://tusharma.in/preprints/ICSE2026_FlipFlop.pdf. Targeted for ICSE 2026.

[31] Pooja Rani, Jonas Zellweger, Veronika Kousadianos, Luis Cruz, Timo Kehrer, and Alberto Bacchelli. 2024. Energy Patterns for Web: An Exploratory Study. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Society* (Lisbon, Portugal) *(ICSE-SEIS'24)*. Association for Computing Machinery, New York, NY, USA, 12–22. https://doi.org/10.1145/3639475.3640110

[32] Nurzihan Fatema Reya, Abtahi Ahmed, Tashfia Rifa Zaman, and Md. Motaharul Islam. 2023. GreenPy: Evaluating Application-Level Energy Efficiency in Python for Green Computing. *Annals of Emerging Technologies in Computing* (2023). https://api.semanticscholar.org/CorpusID:259509821

[33] Ana Ribeiro, João F. Ferreira, and Alexandra Mendes. 2021. EcoAndroid: An Android Studio Plugin for Developing Energy-Efficient Java Mobile Applications. In *2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 62–69. https://doi.org/10.1109/QRS54544.2021.00016

[34] S Shivadharshan, P Akilesh, Rajrupa Chattaraj, and Sridhar Chimalakonda. 2024. CPPJoules: An Energy Measurement Tool for C++. *CoRR* abs/2412.13555 (2024). https://doi.org/10.48550/ARXIV.2412.13555

[35] Marina Sokolova and Guy Lapalme. 2009. A systematic analysis of performance measures for classification tasks. *Information Processing Management* 45, 4 (2009), 427–437. https://doi.org/10.1016/j.ipm.2009.03.002

[36] C. Spearman. 1904. The Proof and Measurement of Association Between Two Things. *American Journal of Psychology* 15 (1904), 88–103.

[37] Weisong Sun, Chunrong Fang, Yun Miao, Yudu You, Mengzhe Yuan, Yuchen Chen, Quanjun Zhang, An Guo, Xiang Chen, Yang Liu, and Zhenyu Chen. 2023. Abstract Syntax Tree for Programming Language Understanding and Representation: How Far Are We? arXiv:2312.00413 [cs.SE] https://arxiv.org/abs/2312.00413

[38] John W. Tukey. 1977. *Exploratory data analysis*. Addison-Wesley Pub. Co., Reading, Mass.

[39] Roberto Verdecchia, Luís Cruz, June Sallou, Michelle Lin, James Wickenden, and Estelle Hotellier. 2022. Data-Centric Green AI: An Exploratory Empirical Study. *CoRR* abs/2204.02766 (2022). https://doi.org/10.48550/ARXIV.2204.02766

[40] Fadi Wedyan, Rachael Morrison, and Osama Sam Abuomar. 2023. Integration and Unit Testing of Software Energy Consumption. In *2023 Tenth International Conference on Software Defined Systems (SDS)*. 60–64. https://doi.org/10.1109/SDS59856.2023.10329262

[41] Niklaus Wirth. 1995. A Plea for Lean Software. *Computer* 28, 2 (1995), 64–68.

[42] Claes Wohlin, Per Runeson, Martin Hst, Magnus C. Ohlsson, Bjrn Regnell, and Anders Wessln. 2012. *Experimentation in Software Engineering*. Springer Publishing Company, Incorporated.

[43] Włodzimierz Wysocki, Ireneusz Miciuła, and Przemysław Plecka. 2025. Methods of Improving Software Energy Efficiency: A Systematic Literature Review and the Current State of Applied Methods in Practice. *Electronics* 14, 7 (2025). https://doi.org/10.3390/electronics14071331

[44] Thomas Zaragoza, Adel Noureddine, and Ernesto Exposito. 2025. A systematic mapping study on software-based feedback for energy consumption. *Renewable*

[5] https://doi.org/10.5281/zenodo.18366913

and Sustainable Energy Reviews 222 (2025), 115889. https://doi.org/10.1016/j.rser.2025.115889